

Improved Pre-processing Techniques for Mobile-based OCR Systems

Nikesh D.C.

Department of Electronics &
Computer Engineering
Pulchowk Campus, I.O.E
Tribhuvan University, Nepal
075bct052.nikesh@pcampus.edu.np

Ravi Pandey.

Department of Electronics &
Computer Engineering
Pulchowk Campus, I.O.E
Tribhuvan University, Nepal
075bct065.ravi@pcampus.edu.np

Rohan Chhetry.

Department of Electronics &
Computer Engineering
Pulchowk Campus, I.O.E
Tribhuvan University, Nepal
075bct066.rohan@pcampus.edu.np

Bibha Sthapit

Department of Electronics &
Computer Engineering
Pulchowk Campus, I.O.E
Tribhuvan University, Nepal
bibha@ioe.edu.np

ABSTRACT

Abstract—With the growing prevalence of mobile devices and the increasing need for on-the-go document processing, mobile-based Optical Character Recognition (OCR) has gained significant attention. However, the accuracy and robustness of OCR systems also rely heavily on the quality of pre-processing techniques applied to the input document images. This paper presents novel and robust pre-processing techniques specifically tailored for the image in the wild captured using mobile devices that significantly improve the performance of document OCR. By addressing common challenges such as variability in lighting conditions, image noise, and varieties of layouts across the spectrum of documents, the proposed methods aim to enhance the accuracy and efficiency of OCR systems, enabling more reliable document digitization of the images captured using mobile devices.

Index Terms—Document pre-processing, mobile-based OCR, binarization, noise removal, reading order determination

I. INTRODUCTION

In recent years, mobile devices have become an integral part of our daily lives, offering convenient access to information, communication, and productivity tools. With their ever-increasing computational power and advanced imaging capabilities, these devices have also transformed into powerful tools for document processing tasks. Optical Character Recognition (OCR) stands out as one of the key applications that enables users to extract text from images or scanned documents to allow for a more editable and searchable format. While OCR technology has seen substantial advancements over the years, its implementation on mobile devices presents unique challenges. Limited computational resources, varying image quality, and the need for real-time processing pose constraints that demand specialized pre-processing techniques. The pre-processing stage plays a crucial role in OCR accuracy, as it significantly affects the subsequent text recognition phase. Therefore, improving pre-processing techniques tailored specifically for mobile-based OCR systems is of paramount importance.

This paper presents a set of improved pre-processing techniques designed to address the challenges encountered

in mobile-based document OCR. Though the term "pre-processing" is usually used to refer to image improvement methods, in this paper the term is used loosely for the methods used before the main text recognition process. The methods proposed aim to enhance the quality of document images by mitigating common issues such as inconsistency and noise in document images caused by variability in lighting conditions, stains, or simply the texture of paper upon which the text is printed or written, and varieties of layouts across the spectrum of documents such as books, articles, magazines, newspaper and so on. By optimizing these pre-processing steps, we strive to achieve higher accuracy and robustness in OCR results, ultimately enhancing the overall user experience when digitizing documents on mobile platforms.

In this paper, we propose three methods: (1) A binarization method with improvement over Sauvola's thresholding method [1], (2) Noise removal using connected component analysis, and (3) reading order determination based on projection profiles [2].

The remainder of this paper is organized as follows: Section 2 provides an overview of methods that have inspired the proposed preprocessing techniques. Section 3 discusses the related works that have been done on mobile-based OCR systems. Similarly, Section 4 describes the challenges specific to mobile-based OCR systems and those that are tackled while developing the proposed system. In Section 5, we present our proposed preprocessing methods in detail, highlighting their contributions. Section 6 describes the experimental setup and evaluation metrics employed to assess the effectiveness of our techniques and summarizes the obtained results and analysis. Finally, Section 7 concludes the paper, by highlighting the contributions and hinting potential avenues for future research.

II. OVERVIEW OF PREPROCESSING TECHNIQUES

Wide varieties of methods have been implemented for extracting texts from image documents under OCR technologies. These methods vary in their intended use conditions, performance, and so on. The process followed by most OCR technologies can be broadly divided into three steps: pre-processing, segmentation, and classification/identification. Pre-

processing can have a great influence on the final OCR result [3]. It usually involves several methods and finally produces a binarized output of the less informative input image, having

minimal non-textual noises.

A. Binarization

To extract text from image documents, it is often sufficient to represent the images in grayscale, as the color information is not necessary for text extraction. Just for text extraction, a binary image is sufficient. A binary image is one where text and background are represented by two distinct values, usually 1 and 0, respectively. Binarization is the process of converting a grayscale (or colored) image into the corresponding binary image. This is typically achieved through thresholding, which involves comparing the pixel intensities with a threshold value. Thresholding can be performed globally or locally. Global thresholding applies a single threshold value to all pixels, while local thresholding uses separate thresholds for each pixel or region. Local thresholding is generally more effective as it takes into account local variations such as spot illuminations however also consumes more time.

1) Otsu's Thresholding: Otsu's thresholding [4] is a widely applied global thresholding method based on the analysis of the image's pixel-level histogram. The objective is to find a threshold that maximizes the between-class variance, effectively separating the foreground and background pixels. The interclass variance is given by the equation

$$\begin{aligned}\sigma_B^2 &= w_0(\mu_0 - \mu_T)^2 + w_1(\mu_1 - \mu_T)^2 \\ &= w_0w_1(\mu_1 - \mu_0)^2\end{aligned}\quad (1)$$

w_0 is the proportion or probability of the first class and w_1 is the proportion or probability of the second class and are given by

$$w_0 = \sum_{i=0}^L p_i = \omega(k) \quad (2)$$

$$w_1 = \sum_{i=k+1}^L p_i = 1 - \omega(k) \quad (3)$$

Here p_i is the probability of occurrence of i^{th} level and n_i is the number of pixels at level i

$$p_i = n_i/N, p_i \geq 0, \sum_{i=0}^L p_i = 1 \quad (4)$$

μ_T is the mean level of the entire image, μ_0 is the mean level of first class and μ_1 is the mean level of the second class

$$\mu_0 = \sum_{i=0}^L ip_i / w_0 = \mu(k)\omega(k) \quad (5)$$

$$\mu_1 = \sum_{i=k+1}^L ip_i / w_1 = \frac{\mu(T) - \mu(k)}{1 - \omega(k)} \quad (6)$$

$$\mu_T = \sum_{i=0}^L ip_i \quad (7)$$

The optimal threshold T is given as:

$$\sigma_B^2(T) = \max_{1 \leq k < L} \sigma_B^2(k) \quad (8)$$

However, Otsu's thresholding is not adaptive to changing image conditions and performs poorly in the presence of irregular shadows or illumination variations.

2) Sauvola's thresholding: Sauvola's thresholding [1] is a well-known local thresholding method that calculates the threshold for each pixel based on the mean and variance of a window centered around that pixel. Sauvola's thresholding is an improvement over Niblack's method [5]. The formula for thresholding using Sauvola's threshold is given as follows.

$$T = m * (1 + k * (s/R - 1)) \quad (9)$$

Here T is the calculated threshold for the pixel 'p' under consideration, 'm' is the mean, and 's' is the standard deviation of pixels centered around the pixel 'p' around a size of window 'w'. R is the dynamic range of standard deviation taken generally as 128 for an 8-bit depth image. 'k' is a parameter whose value is usually determined from experimentation. The window size 'w' and 'k' are the two parameters that affect threshold values and thus require further consideration. Sauvola's method is sensitive to these parameters and performs

better when properly tuned. An adaptive approach, like using

multiple windows of different scales, can account for variations in text sizes across the image and improve binarization results [6]. Further, the parameter 'k' plays an even more significant role in the final output, and the optimal value for 'k' seems to change depending on image conditions. However, most papers use a value in the range between 0.33 to 0.5 based on experimentation [1] for required purposes. The parameter value that is good for one kind of image may not prove useful for others so the naive algorithm is not robust for different kinds of document images. There is a lack of proper and

conclusive research on this matter.

B. Noise Reduction

After binarization, the resulting image may contain noise, which can affect the accuracy of OCR. Noise in binarized text images often appears as salt and pepper noise or marginal noise. Salt and pepper noise manifests as scattered patches, while marginal noise arises from the separation of text documents and the background in the original image.

1) Morphological Operations: Morphological operations, such as erosion (shrinking) and dilation (growing), can be used to remove noise from binarized images [7]. A combination of these operations, such as opening (erosion followed by dilation) or closing (dilation followed by erosion), can help reduce noise. However, these methods may not be suitable when dealing with small characters, as they can remove or distort important information.

2) **Border Noise Removal:** Binarized images may contain border noise, which includes undesired or cropped text from neighboring pages or non-textual elements from photos or background surfaces. Border noise removal techniques can be employed to mitigate these issues. One approach [8] involves using a Gaussian filter to obtain a smoothed image, followed by text line detection using ridge detection algorithms. Textual noise ridges near the image borders can be filtered out, and the remaining ridges can be used to determine the left and right boundaries of the text region. Topmost and bottom-most ridges can define the page boundaries for multi-page.

3) **Component Filtering:** A component-based filtering technique can be used to classify individual components obtained by connected component analysis [9] as text or non-text components. The non-text components can then be discarded as unwanted noise. To filter the components, each component can be evaluated based on its properties such as border proximity, count of black pixels, the density of black pixels in bounding rectangle, relative size of the component with respect to the image, and several other similar metrics. The count of black pixels can help filter extremely small components as salt and pepper noise since a text character image needs to have a certain minimum number of pixels for it to be recognizable as a character. Some of the noise is seen in the form of diagonal streaks of black pixels which form a large bounding rectangle, thus having a very low density of black pixels. Marginal noise is often seen as large rectangular patches of dark pixels caused by image background or page borders; the height and width of components are used to decide if it is noise or text. A combination of these metrics is used with respective score thresholds to classify a component as a text or non-text.

C. Layout Analysis

Layout analysis involves understanding the spatial arrangement of the document. This involves segmenting the page into meaningful units such as paragraphs, headings, captions, and tables, as well as identifying the hierarchical structure and relation among these units. One of the key challenges in layout analysis is the inherent variability and complexity of document layouts. Documents can exhibit diverse styles, fonts, sizes, and orientations, making it difficult to establish consistent rules for layout analysis. Additionally, the presence of graphical elements, noise, and artifacts further complicates the process. Layout analysis methods are usually classified as top-down and bottom-up based on the approach taken for segmenting the page into its corresponding units.

III. RELATED WORKS

Numerous methods have been implemented targeted at mobile-based OCR (Optical Character Recognition) systems. For instance, Chang and Ying [10] introduced a technique aimed at identifying regions of interest defined by four corners to locate signs in natural scenes. They also utilized Otsu's thresholding method to convert grayscale images into binary format before employing the Tesseract OCR engine for image-to-text conversion. Similarly, Bhaskar et al. [11] developed

an Android application designed to extract information from business cards. In this application, images captured by Android devices were divided into a 2x3 block array and thresholded using MATLAB's "graythresh" command, which internally employs Otsu's method. They further delineated the card's outline through morphological dilate and erode operations, followed by a Hough transform. A perspective transformation based on the standard card dimensions was applied, followed by a second round of adaptive thresholding and vertical segmentation. Finally, the image was subjected to Tesseract OCR for text extraction. Sourvanos and Tsatiris [12] conducted experiments involving natural occurrences of textual information in various real-world conditions, including challenging scenarios with low light. Document images were captured using mobile devices and a series of image-processing steps were employed. Initially, the image underwent binarization using adaptive thresholding in small regions. Subsequently, various techniques, such as median filtering, Gaussian blurring, morphological closing and opening operations, contrast enhancement, and non-local means denoising, were applied to remove noise and enhance the image. Additionally, Singh and Sharma [13] discussed various preprocessing techniques tailored for images captured by mobile devices. Their approach encompassed projection profile analysis, Hough transform, and nearest neighbor analysis for skewness detection and correction, Gaussian and median filtering for noise reduction, Canny edge detection for cropping and perspective correction, as well as fixed thresholding and the use of methods like Otsu's, Niblack's, and Sauvola's for binarization.

IV. CHALLENGES IN MOBILE OCR

A fundamental and frequently encountered challenge in the realm of vision-based systems lies in coping with the intricacies inherent to various scenes. These complexities stem from a multitude of factors. Notably, the presence of background clutter and noise often introduces disruptive elements, confounding the character recognition system's ability to distinguish between textual and non-textual content, leading to erroneous classifications. Additionally, lighting conditions wield a significant influence over the outcomes of OCR [12]. Cutting-edge techniques in this field prioritize achieving consistency in diverse lighting scenarios. In the realm of mobile and "in the wild" applications, issues stemming from out-of-focus or blurred images can also pose formidable hurdles. Perspective distortions and skewed images further compound the challenges faced by mobile OCR applications [13]. Another noteworthy issue in OCR systems pertains to their adaptability across a wide spectrum of document types and layouts. Beyond these image-related predicaments, mobile-based systems grapple with inherent computational limitations. To surmount these computational constraints, the adoption of a client-server architecture, wherein processing is offloaded to potent server computers, becomes an option. However, this necessitates an active network connection for the system to operate, introducing potential network-related issues.

In the context of this research, particular attention has been directed toward addressing image degradation resulting from uneven illuminations and shadows, as well as mitigating noise introduced by variations in document-paper textures and page borders. Simultaneously, the goal has been to develop a system adaptable to a broad array of document layouts while optimizing processing speed. Additionally, a key objective has been the creation of an automated, fully functional system that requires minimal user interaction. Such constraints hold potential utility in applications designed for the visually impaired and in the development of fully automated systems.

V. METHODOLOGY

A real-time offline Android application was developed implementing the pre-processing methods along with Tesseract OCR using C# and Unity game engine due to the familiarity of the authors with the environment. The whole OCR process is completed in about 3-4 seconds. Similarly, an online application was also developed with the client-server model using React Native for the client application and Python and Java for the server application, with the server handling all the processing and the client only responsible for sending the captured image in grayscale and playing audio sent by server converted from the OCR result. The limiting factor in this application was observed to be the networking infrastructure. The proposed improved pre-processing techniques for binarization, noise removal, and reading order determination are presented as follows.

A. Binarization

The proposed binarization procedure is based on some improvements over Sauvola's thresholding algorithm. Sauvola's algorithm was improved by using integral image [14] for fast computation, unlike interpolation as proposed in the original paper [1]. The integral image method allows for finding the sum of any sized window of an image in a constant number of operations. Similarly, a second improvement to the algorithm is the use of a dynamic value for 'k' instead of a predefined one. This makes the algorithm more robust to variations in image conditions. In our experiments, a simple regression model was used that determines the parameter values dynamically for each document image based on some feature extracted from the image that may affect the value parameters to use. Thus selected features need to provide some

measure of the document image conditions appropriate for Sauvola's thresholding method. The improvements are further

discussed in the subsections after. As for the window size for thresholding, it was taken as a tenth of the minimum between height and width.

1) Execution time Improvement: Since most local thresholding methods, in particular, Sauvola's thresholding requires calculating mean and variances for window centered around every pixel, performance can be improved by using integral image [14] which allows calculation of sum and hence mean using a constant amount of arithmetic operations. Extending this concept, for calculating variance, we can store the integral

image of squares of each pixel value which along with the integral image for the original image can be used to calculate the variance within any size of window. Thus used operations are constant i.e. 4 addition/subtraction and a division operations for mean calculation irrespective of the size of the window. A similar result holds for the variance calculation.

2) Dynamic parameter: A simple regression model was used to predict proper, possibly optimum, value for each document image captured using a mobile phone. For this, a data set consisting of images and their corresponding optimal values for 'k' was required. The optimum value calculation further requires the ground truth of the image and an evaluation metric to determine the similarity of the output result with the ground truth. The optimum value calculation is then done by trying out all possible values in the range 0 to 1 using small increments (in our experiments: 0.05) and then selecting the one that gives the best similarity score. However, there is no need to try all values as increasing the values of 'k' tends to decrease the threshold, effectively removing more noise pixels (and text as well), so, once the similarity score reaches a peak, the optimum value has been found. To evaluate the resemblance of the binarized image with its ground truth we used the F1 score (with text-pixel or '1' as the positive case and background pixel or '0' as the negative case) as accuracy can be misleading [15]. Thus, we require a data-set consisting of image and ground-truth pair, which we obtained from DIBCO(2009-2018) [16]; a total of 116 images and their ground-truths. The regression model requires feature(s) describing the image condition to evaluate a proper value of 'k' for the image. For this a range of measures that were used in the threshold evaluation using Otsu's method [4] such as Otsu's optimal threshold(σ_T), mean of the first class(μ_0), mean of second class (μ_1), variance of first class (σ_0), variance of second class (σ_1), and other simple global measures such as mean (m), standard deviation (sd), entropy(E), and their combinations using multiplication, division and power such as sd/m , sd/σ_T , and so on representing the image as a whole were selected as possible candidates for the measures of image conditions. Then based on correlation between the features and optimal-k values, possible features were selected as being the most effective ones for predicting optimal-k value. The final regression model used for predicting the optimal value of k for the given document image is:

$$k = 0.15 \times \frac{\sigma_1}{\mu_0} + 0.025 \times E - 0.117 \quad (10)$$

Though the given approach focuses on describing the whole image through these features, smaller regions can be selected for better results with a trade-off of execution time. The training however can be done with global feature selection as local feature selection requires a large amount of time.

B. Noise Reduction

The noise reduction is performed after obtaining the binarized image. The noise present in text images that we will be dealing with is mostly scattered patches of salt and pepper

noise throughout the image caused due to uneven shadows in the original image, and the marginal noise as a result of background separation from the text document. In order to remove such noise, we apply the following approaches.

1) Morphological operation: Salt and pepper noise can be largely removed using a morphological open operation followed by a close operation. Window size of 3 was seen best for most cases in our experiments as windows of greater size tend to introduce more distortions in addition to removing the noise. However, for low-resolution images these operations are not useful and rather tend to introduce more artifacts, instead reducing the accuracy of text recognition. In such cases, component filtering is more appropriate albeit at a possibly larger computation cost. For our purpose, morphological open operation followed by morphological close operation was conditionally performed only when the image width and height were greater than 1000 pixels.

2) Component Filtering: The proposed method for noise removal using connected component analysis is as follows. First, the bordering white space is removed from the image such that all the image borders, at the top, bottom, left, and right touch a text pixel labeled by the binarization procedure (that may actually be a text or noise pixel). Then the bordering components (that are touching or very close to the image border) are evaluated based on the height and width of the bounding box enclosing each component. A conservative estimate of 0.2 times height or width is taken as the criteria for labeling the component as noise or not. That is, if the height or width of the bordering component is larger than 20% of the image height or width respectively then the component is filtered out as being noise. Similarly, a second pass of filtering is done using the same procedure to remove more noise that may have been obscured by a large bordering noise. The passes may be repeated until no more noise is detected but for our experiment only two passes were used to limit computation expense. A post-noise removal is performed after having more accurate information on average character size from the process of reading order determination. The reading order determination method below usually segments the image into smaller sub-images having almost uniform size which helps to make more accurate and well-informed decisions during component filtering; the components above five times the average width or height are filtered out as noise. Unlike the flatbed scanner systems, the image captured by mobile devices tends to capture the background in addition to the region of interest. This introduces marginal noise and blobs which are effectively removed by this method.

C. Reading Order Determination

The proposed reading order determination method has a lenient constraint compared to most layout analysis methods [17] in that it only gives proper reading order for sub-images but the sub-images itself may or may not correspond directly to document layouts in a uniform scale. For instance, one sub-image may be segmented at the word level but others may only be segmented at paragraph level. However, further information

can be used for other purposes that require such precision. The main goal of layout analysis for our purpose was simply to determine the proper reading order of texts which can be helpful for OCR engines such as Tesseract [18]. For this, we don't require the full sophistication of layout determination. In most of the documents, white spaces are used to separate various sections of text and the white spaces are proportional in that large white spaces usually signify more important separations. Since this observation is consistent across varieties of textual information, the method itself is robust towards variations in document layouts. Taking this observation into account, the proposed reading order determination method is developed as described below.

- 1) First any bordering white-spaces are removed from the document image
- 2) Then the white space that spans along the whole document's height or width is found.
- 3) Next the image is separated into two 'sub-images' based on white-space as the separator.
- 4) If multiple such white spaces spanning the whole document's height or width exist then the largest one is selected for bi-parting the image into two sub-images.
- 5) The reading order is as follows: If the image is separated horizontally then read the top sub-image first then the bottom sub-image, else if the image is separated vertically then read the left sub-image first then the right sub-image.
- 6) The aforementioned processes are recursively applied until the required level of segmentation is reached.

This separation structure can be implemented as a binary tree with the root representing the entire document, each node representing a rectangular bound complementary to its sibling, and the leaves representing the final segmentation. For white-space detection projection profile method [2] was used with some margin in threshold (instead of zero sum over projection) to account for possible noise. The required level of segmentation may vary depending on applications. For instance, while testing with Tesseract OCR in our experiments, separating different columns of text was enough as the rest of the texts are in natural order. Similarly, skew and perspective correction methods such as [19] may be required as the proposed method uses projection profiles. Using a binary tree instead of a n-ary tree as in XY-cut [20] can make the algorithm more robust as it is less likely to consider coincidental matching of white-space separation of lines of different columns of text as a higher level separation like that shown in figure 1.

VI. RESULT

Tesseract OCR was used for most part of the experimentation as it is widely used in many systems.

A. Binarization

By use of Integral image a huge performance improvement, especially for large window size, was achieved as shown in figure 2. The execution time also includes the time taken to create the integral image. As can be observed the time



Fig. 1. Coincidental matching of separation of different columns shown in red

taken while not using integral image computation increases in squared fashion over window size. This can be confirmed by the fact that the number of iterations or pixels increases in squared power when window size (height or width) increases. However, with an integral image, the execution time remains almost constant.

For the dynamic parameter, a data set of 116 different images from DIBCO(2009-2018) was used. They were split randomly into nearly equal test and training samples of 53 and 63 respectively. An average f1 score of 0.84 compared to 0.76 while using a constant value of 0.35 for the 'k' parameter was observed. Further, a performance as low as 0.25 f1-score was observed while using constant value whereas while using that predicted by the regression model, the lowest performance was 0.52 f1-score. Other statistics are compared in table I. This suggests that although the simple regression model is far better and more stable than simply using a constant value from experimentation for the kinds of documents under consideration, the performance can be improved by increasing the data set size and model complexity appropriately. Also, the parameter 'k' in Sauvola's thresholding will be better if

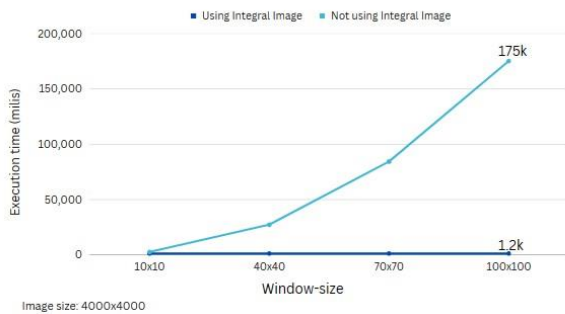


Fig. 2. Speed Improvement with integral image

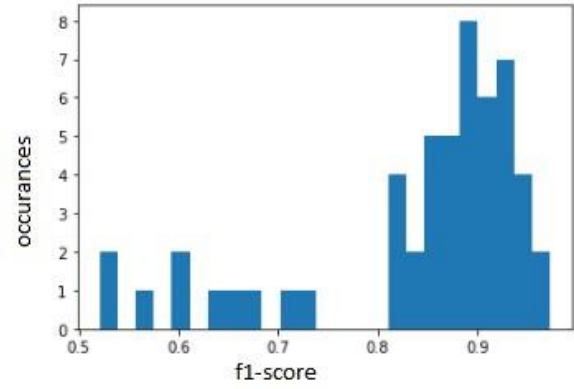


Fig. 3. F1 scores over test set using dynamic parameter

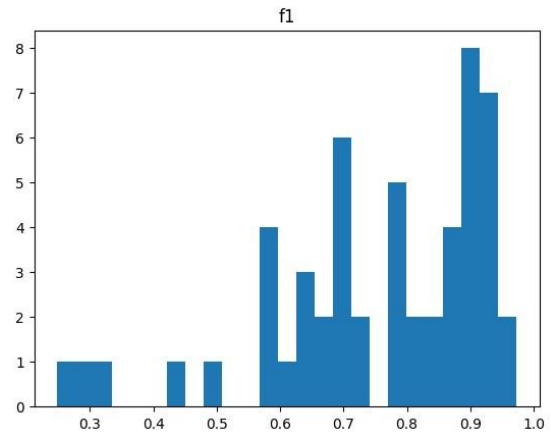


Fig. 4. F1 scores over test set using a constant value for 'k'

replaced by some other factors rather than just a constant value. Those factors may be obtained by using measures from global thresholding methods such as Otsu's thresholding as highlighted in this paper. The distribution of f1scores is shown by taking a histogram with intervals of 0.04 in figure 3 and 4. We also experimented to see if the value of the dynamic range 'R' has any effect on the output of binarization. For this, a comparison between the fixed value of 128 and the value calculated as half of the range between the minimum and maximum values of pixels present in the image was done. The result suggested no significant difference between the two.

Statistics	Sauvola's method	Improved method
Mean	0.757	0.843
Standard Deviation	0.175	0.116
Minimum	0.248	0.521
Maximum	0.972	0.973
25th percentile	0.676	0.828
50th percentile	0.792	0.883
75th percentile	0.901	0.917

TABLE I
COMPARISON OF F1-SCORES OF SAUVOLA'S METHOD WITH THE IMPROVEMENT USING DYNAMIC PARAMETER OVER TEST SAMPLES



Fig. 5. Segmentation with reading order for a magazine

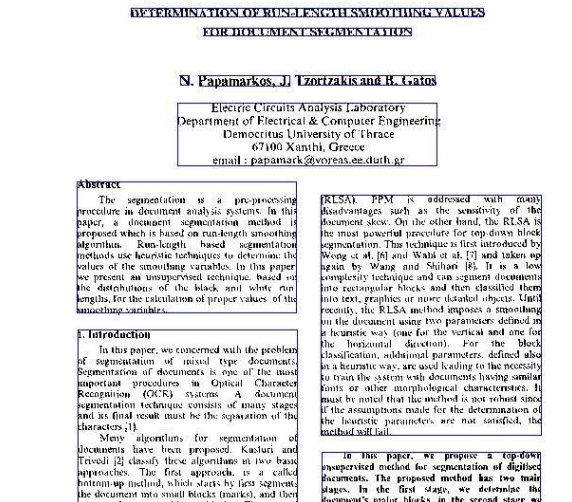


Fig. 6. Segmentation with reading order for an article

B. Noise Reduction

The component filtering technique is able to remove the noise present in the binarized image to a considerable extent. In order to test the accuracy improvement by noise reduction, we calculated the CER (Character Error Rate) [21] for the output text generated for the binarized image and ground truth image and then for the denoised image and ground truth image. Tesseract OCR was used for actual text extraction. An improvement was seen in the CER after reducing noise by filtering out the non-text components as OCR systems tend to infer noisy blobs as random text characters.

C. Reading Order Determination

The aforementioned technique for reading order is able to cope with varying layouts such as that of articles and magazines and the sub-images or bounding boxes generated by it for those documents are shown in figure 5 and figure 6. This method however may require skew correction and non-textual component filtering for more robust operation.

D. OCR Accuracy Improvement

A comparison of only using Tesseract OCR and Tesseract OCR in conjunction with the proposed methods based on CER is presented in the figure 7. A sample of 14 images were evaluated for the accuracy comparison. Some sample images used for the comparison of OCR accuracy are shown in figure 8. These were captured using a mobile device and highlight some common problems such as shadows that are quite common in applications for the visually impaired. As shown by the graph the proposed system performs significantly superior to only using the Tesseract system. This can be attributed to the fact that Tesseract OCR on its own employs Otsu's method of binarization; a global method that cannot cope properly with changing image conditions such as uneven illumination and shadow. The proposed system utilizes Sauvola's thresholding; a local method and improvement over

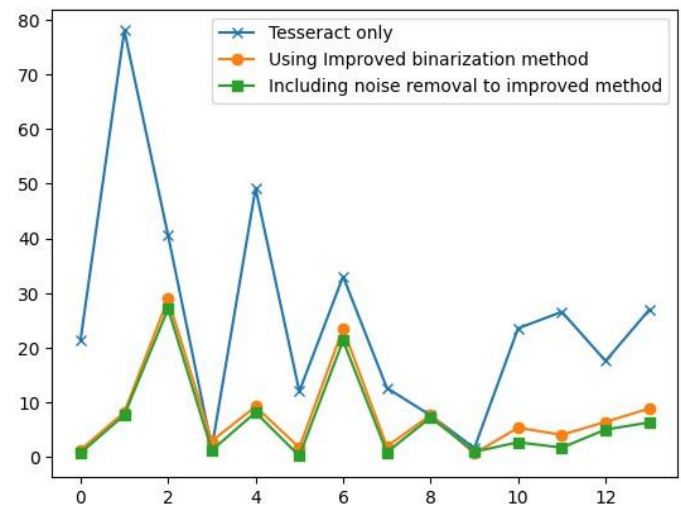


Fig. 7. OCR improvement in terms of CER

it using a prediction model for the 'k' parameter from some image properties extracted from Otsu's method. The selection of a proper binarization method thus has a great impact on final OCR accuracy. The noise reduction process also removes large blob-like elements and small salt-pepper noise enhancing the result further.

VII. CONCLUSION

In this paper, we proposed improved and robust pre-processing techniques specifically tailored for mobile-based Optical Character Recognition (OCR) systems. The aim was to improve the accuracy and efficiency of document OCR by addressing common challenges encountered in mobile document imaging, such as variability in lighting conditions, image noise, and varieties of layouts across different documents. The proposed methods can be used in conjunction

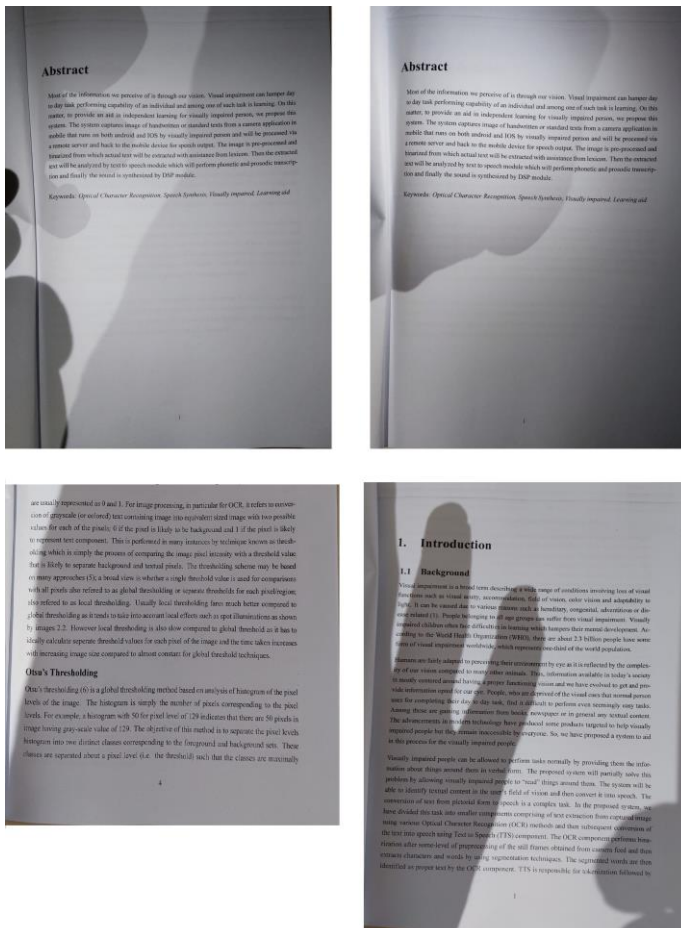


Fig. 8. Some sample images used for OCR evaluation

with OCR systems such as Tesseract to significantly improve text extraction accuracy with minimal computational time overhead. Applying two-pass noise removal before and after segmentation can lead to more accurate results. First noise removal removes marginal noise for the image as a whole which can benefit segmentation and the segmentation in turn can provide more accurate information on character size to the second pass noise reduction process to remove non-textual artifacts. This paper highlights that measures used in global thresholding methods can represent image conditions and thus be used as a feature for predicting dynamic parameter values in local adaptive methods. Similarly, the use of integral images can significantly decrease the computational time for adaptive methods requiring sum operations like mean and variance calculation over windows of image.

REFERENCES

- [1] J. Sauvola and M. Pietikainen, "Adaptive document image binarization," *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, 2000.
- [2] B. C. M. Javed, P. Nagabhushan, *Extraction of Projection Profile, Run-Histogram and Entropy Features Straight from Run-Length Compressed Text-Documents*. India: Computer Vision and Pattern Recognition, 2008.
- [3] M. Sezgin and B. Sankur, "Survey over image thresholding techniques and quantitative performance evaluation", *Journal of Electronic Imaging*, Vol 13, pg 146-168, 2004
- [4] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS*, VOL. SMC-9, NO. 1, pp. 62–66, 1979.
- [5] W. Niblack, "An Introduction to Digital Image Processing". Englewood Cliffs: Prentice Hall, 1986.
- [6] G. Lazzara and T. Géraud, "Efficient multiscale sauvola's binarization," *International Journal on Document Analysis and Recognition*, pp. 105–123, 2014.
- [7] N. Jamil, T. Sembok, and Z. Bakar, "Noise removal and enhancement of binary images using morphological operations," *2008 International Symposium on Information Technology*, vol. 3, pp. 1 – 6, 09 2008.
- [8] S. S. Bukhari, F. Shafait, and T. M. Breuel, "Border noise removal of camera-captured document images using page frame detection," in *Camera-Based Document Analysis and Recognition* (M. Iwamura and F. Shafait, eds.), (Berlin, Heidelberg), pp. 126–137, Springer Berlin Heidelberg, 2012.
- [9] B. Donald and J. Christopher, "Single pass connected components analysis", *Proceedings of Image and Vision Computing*, 2007.
- [10] L. Z. Chang and Z. Z. Ying, "Robust Pre-processing Techniques for OCR Applications on Mobile Devices", *Mobility '09: 6th International Conference on Mobile Technology, Application & Systems*, pp 1-4, 2009.
- [11] S. Bhaskar, N. Lavassar and S. Green, "Implementing Optical Character Recognition on the Android Operating System for Business Cards", 2010.
- [12] N. Sourvanos and G. Tsadiris, "Challenges in input preprocessing for mobile OCR applications: A realistic testing scenario", *2018 9th International Conference on Information, Intelligence, Systems and Applications (IISA)*, 2018.
- [13] P. Singh and D. V. Sharma, "Pre-Processing of Mobile Camera Captured Images for OCR", *International Journal of Intelligent Systems and Applications in Engineering*, pp.147-155, 2023.
- [14] F. C. Crow, "Summed-area tables for texture mapping," 1984.
- [15] M. Sokolova, N. Japkowicz, and S. Szpakowicz, "Beyond accuracy, f-score and roc: A family of discriminant measures for performance evaluation," *AI 2006: Advances in Artificial Intelligence, Lecture Notes in Computer Science*, vol. 4304, pp. 1015–1021, 2006.
- [16] I. Pratikakis, K. Zagoris, and I. Santaniello, "Icdar 2019 competition on document image binarization (dibco 2019)," *2019 International Conference on Document Analysis and Recognition (ICDAR)*, 2017.
- [17] R. Cattoni, T. Coianiz, S. Messelodi, and C.M. Modena, "Geometric Layout Analysis Techniques for Document Image Understanding: a Review", *ITC-IRST*, 2000.
- [18] S. R., "An overview of the tesseract ocr engine", *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, 2007.
- [19] F. Guo, Y. Li and P. Liu, "A fast page outline detection and dewarping method based on iterative cut and adaptive coordinate transform", *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*, IEEE, 2019.
- [20] J. Ha, R.M. Haralick and I.T. Phillips, "Recursive X-Y Cut using Bounding Boxes of Connected Components", *2019 International Conference on Document*.
- [21] C. Neudecker, K. Baierer, M. Gerber, C. Clausner, A. Antonacopoulos, and S. Pletschacher, "A survey of ocr evaluation tools and metrics," 2021.